

BASES DE DATOS PL/SQL



ESCUELA DE INGENIERÍA INFORMÁTICA

Características

- ✎ Algunas características del lenguaje:
 - No es sensible a mayúsculas/minúsculas
 - Comentarios hasta fin de línea: --
 - Comentarios multilínea: /*... .. */
 - Literales de cadena se especifican entre comillas simples → 'esto es una cadena'
 - Literales de carácter se especifican entre comillas simples → 'c'
 - Fechas se especifican entre comillas simples siguiendo el formato por defecto del sistema o indicando el formato → '17/11/2008'

Bloque (Anónimo)

[DECLARE

Declaración de Variables;]

BEGIN

Sentencias Ejecutables;

[EXCEPTION

Tratamiento de Excepciones;]

END;

Subprogramas

- ✎ Son bloques PL/SQL con un nombre
 - Pueden recibir parámetros y ser invocados directamente
 - Puede haber varios con el mismo nombre siempre y cuando tengan distinto tipo/número de parámetros
- ✎ Tipos
 - Procedimientos (**PROCEDURE**) -> Llevan a cabo acciones
 - Funciones (**FUNCTION**) -> Devuelven valores
- ✎ Creación
 - Es necesario tener el privilegio del sistema de **CREATE PROCEDURE**
 - Si se crea en un esquema diferente al propio es necesario el privilegio del sistema **CREATE ANY PROCEDURE**
- ✎ Invocación
 - Desde otro subprograma
 - Desde aplicaciones
 - Desde SQL+ (**EXECUTE**)

Subprogramas - Parámetros

- ✎ Los subprogramas pueden recibir y devolver parámetros:
 - **Parámetros de entrada (IN)**, se recibe un valor. Es la opción por defecto
 - **Parámetros de salida (OUT)**, se devuelve un valor
 - **Parámetros de entrada y salida (IN OUT)**, se recibe un valor y se puede devolver un valor
- ✎ Los parámetros pueden tener valor por defecto, de tal manera que en la invocación al subprograma no sea necesario pasar el parámetro

Subprogramas - Parámetros

∞ Ejemplos:

- ∞ **Subpro1 (a NUMBER) ;**--equivalente a subpro1(a IN NUMBER)
- ∞ **Subpro2 (a IN NUMBER:=1) ;**--si no se le pasa toma 1
- ∞ **Subpro2 (a IN NUMBER DEFAULT 1) ;**--igual al anterior
- ∞ **Subpro3 (a OUT DATE) ;**--devuelve una fecha
- ∞ **Subpro4 (a IN OUT NUMBER) ;**--recibe y devuelve valor
- ∞ **Subpro5 (a IN NUMBER, b IN NUMBER :=4) ;***/se le puede invocar
subpro5(33) ; subpro5(33,6) ; */
- ∞ **Subpro6 (a IN NUMBER, b OUT VARCHAR2) ;**/*en los varchar2 no
se indica el tamaño*/

Procedimientos

∞ Sintaxis

```
CREATE [OR REPLACE] PROCEDURE [esquema.]nombre  
  [(argumento 1 [{IN|OUT|IN OUT}] tipo_dato [{:=|DEFAULT}  
  valor],  
  .....  
  argumento n [{IN|OUT|IN OUT}] tipo_dato [{:=|DEFAULT}  
  valor]))  
  
{IS|AS}  
  cuerpo de procedimiento
```

Procedimientos

∞ Estructura

```
CREATE OR REPLACE PROCEDURE nombre IS  
    /*sección declarativa (opcional) (sin DECLARE)*/  
BEGIN  
    /*sección ejecutable (obligatoria)*/  
    [EXCEPTION]  
    /*sección de excepciones*/  
END [nombre];
```


Procedimientos

∞ Ejemplo

```
CREATE OR REPLACE PROCEDURE ejemplo (  
    p_departamento NUMBER, p_emp temp.empno%TYPE,  
    p_accion NUMBER(1) DEFAULT 1 )  
IS  
    v_aux NUMBER;  
    v_nuevo_dep tdept.deptno%TYPE;  
  
BEGIN  
    .....  
EXCEPTION  
    WHEN .....  
END ejemplo;
```

Funciones

☞ Sintaxis

```
CREATE [OR REPLACE] FUNCTION [esquema.]nombre  
  [(argumento 1 [{IN|OUT|IN OUT}] tipo_dato [{:=|DEFAULT}  
    valor],  
  .....  
  argumento n [{IN|OUT|IN OUT}] tipo_dato [{:=|DEFAULT}  
    valor]))  
RETURN tipo_dato_retorno {IS|AS}  
cuerpo de función
```

Funciones

- ✎ En el cuerpo de la función es necesario que se use la orden **RETURN** para devolver el valor de retorno de la función
- ✎ (También se puede usar en procedures sin ningún parámetro y sirve para provocar la salida inmediata del mismo. No recomendado)

Funciones

∞ Ejemplo

```
CREATE OR REPLACE FUNCTION calcula (  
    p_dni tpersona.dni%TYPE, p_sal OUT NUMBER)  
RETURN NUMBER  
IS  
    v_porcen NUMBER(3,2) ;  
BEGIN  
    .....  
    IF v_porcen>0.25 THEN  
        RETURN 1;  
    ELSE  
        RETURN 2;  
    END IF;  
END calcula;
```

Subprogramas

✂ Manejo de excepciones

- **RAISE** (para excepciones predefinidas o definidas por el usuario)
- **RAISE_APPLICATION_ERROR** (para crear excepciones con sus propios mensajes de error)
 - RAISE_APPLICATION_ERROR (número, mensaje)
 - El número debe estar entre -20000 y -20999
 - El mensaje puede ser de hasta 512 caracteres

Subprogramas

✎ Privilegio **EXECUTE**

✎ Parámetro **REMOTE_DEPENDENCIES_MODE**

- SIGNATURE
- TIMESTAMP

Tipos de Datos

∞ Más usados

- **NUMBER** [(precisión, escala)]
- **CHAR** [(longitud_máxima)]
- **VARCHAR2** (longitud_máxima)
- **DATE**
- **BOOLEAN**
- **RECORD**
- **TABLE**
- **VARRAY**

Tipos de Datos

∞ Muy importantes:

- Variables y constantes: **%TYPE** y **%ROWTYPE**
- %TYPE declara una variable basada en:
 - El tipo de dato de otra variable
 - El tipo de dato de una columna de una tabla
- Sintaxis %TYPE:
 - <nombre variable%TYPE>;
 - <nombre propietario.tabla.columna%TYPE>;

Tipos de Datos

☞ Muy importantes:

- Variables y constantes: **%TYPE** y **%ROWTYPE**
- **%ROWTYPE** declara una variable registro que representa una fila en una tabla o vista (“variable de tupla”)
 - Las columnas de la fila y los nombres de los campos de la variable registro, tienen los mismos nombres y tipos de datos.
- Sintaxis **%ROWTYPE**
 - `<nombre propietario.tabla%ROWTYPE>;`

Registros

☞ Tipo RECORD:

TYPE nombre IS RECORD

(campo1 {tipo_dato|variable%**TYPE**| tabla.columna%**TYPE**|
 tabla%**ROWTYPE**} [**NOT NULL**],
campo2 {tipo_dato|variable%**TYPE**| tabla.columna%**TYPE**|
 tabla%**ROWTYPE**} [**NOT NULL**], ...
...);

- ☞ Hay que declarar un tipo de dato RECORD y los registros de este tipo
- ☞ Se pueden anidar registros

Registros

∞ Ejemplo

DECLARE

```
TYPE r_emp IS RECORD (  
    numemp NUMBER (4) ,  
    nombreemp VARCHAR2 (10) ,  
    numdept NUMBER (4) NOT NULL :=11) ;  
v_registro r_emp;  
v_registro2 scott.emp%ROWTYPE;
```

BEGIN

```
v_registro.numemp:=1;  
SELECT * INTO v_registro2  
    FROM scott.emp  
    WHERE empno=123;
```

END;

Registros

🌀 Ejemplo

DECLARE

```
TYPE reg_1 IS RECORD (cod NUMBER, descrip VARCHAR2(100));  
TYPE reg_2 IS RECORD (cod NUMBER, descrip VARCHAR2(100));  
v_reg_1 reg_1;  
v_reg_2 reg_2;  
v_reg_3 reg_2;
```

BEGIN

```
v_reg_1.cod:=1;  
v_reg_1.descrip:='ccc';  
v_reg_2.cod:=v_reg_1.cod;  
v_reg_2.descrip:=v_reg_1.descrip;  
--la siguiente línea es incorrecta, no son del mismo tipo  
v_reg_2:=v_reg_1;  
--la siguiente línea es correcta  
v_reg_3:=v_reg_2;
```

END;

Registros

∞ Ejemplo

DECLARE

`v_reg_1 scott.emp%ROWTYPE;`

BEGIN

`SELECT * INTO v_reg_1`

`FROM scott.emp`

`WHERE empno=123;`

`v_reg_1.empno:=343;`

`v_reg_1.sal:=7846;`

`INSERT INTO scott.emp VALUES v_reg_1;`

END;

Registros

- ✎ Los registros no se pueden comparar entre sí (es necesario comprobar la igualdad campo a campo)
- ✎ Los registros no pueden comprobarse para ver si son nulos
- ✎ Las siguientes expresiones son ilegales:
 - `My_Record IS NULL`
 - `My_Record_1 = My_Record_2`
 - `My_Record_1 > My_Record_2`

Declaraciones

Sección DECLARE

- <nombre> [CONSTANT] <tipo de dato> [NOT NULL] [:=<expresión>|DEFAULT <valor>];

Ejemplo

DECLARE

```
impuestos  NUMBER;  
continua   BOOLEAN;  
multiplicador  CONSTANT  NUMBER(3,2)  DEFAULT  0.10;  
fecha      DATE;  
sal_max    NUMBER(11,2) :=500*multiplicador;  
nombre     CHAR(30);
```

BEGIN

Declaraciones

∞ Ejemplo

DECLARE

```
numero_dep dept.deptno%TYPE;  
nuevo_balance NUMBER(8):=10;  
referencia nuevo_balance%TYPE;  
reg_emp emp%ROWTYPE;
```

BEGIN

```
SELECT * INTO reg_emp  
  FROM emp  
  WHERE.....;  
IF reg_emp.deptno=20 THEN
```

```
.....
```

END;

Ámbito y visibilidad de identificadores

```
DECLARE
  a CHAR;
  b NUMBER;
BEGIN
  --a CHAR y b
  DECLARE
    a NUMBER;
    c DATE;
  BEGIN
    --a NUMBER, b y c
  END;
  DECLARE
    d BOOLEAN;
  BEGIN
    --a CHAR, b y d
  END;
END;
```

Asignación y Operadores

∞ Asignación :=

∞ Operadores (mayor a menor precedencia)

**	Exponenciación
+, -	Identidad, negación
*, /	Multiplicación, división
+, -,	Suma, resta, concatenación
=, !=, <, >, <=, >= IS NULL, LIKE, IN, BETWEEN	Comparación
NOT	negación
AND	Unión
OR	Inclusión

Estructuras de control

- ∞ Condicional
- ∞ Iterativo
- ∞ Secuencial

Control condicional

∞ IF THEN

```
IF condición THEN
    conjunto de sentencias;
END IF;
```

∞ IF THEN ELSE

```
IF condición THEN
    conjunto de sentencias1;
ELSE
    conjunto de sentencias2;
END IF;
```

∞ IF THEN ELSIF

```
IF condición1 THEN
    conjunto de sentencias1;
ELSIF condición2 THEN
    conjunto de sentencias2;
ELSE
    conjunto de sentencias3;
END IF;
```

Control condicional

∞ Ejemplos

```
IF x>y THEN  
    alto:=x;  
END IF;
```

```
IF cadena='pe' THEN  
    UPDATE tabla .....;  
ELSE  
    UPDATE tabla2 .....;  
END IF;
```

```
IF contador<20 THEN  
    cambia:=50;  
ELSIF contador>50 THEN  
    cambia:=30;  
ELSE  
    cambia:=20;  
END IF;
```

Control condicional

CASE

```
CASE selector
WHEN selector_value_1 THEN statements_1
WHEN selector_value_2 THEN statements_2
...
WHEN selector_value_n THEN statements_n
[ELSE
  else_statements ]
END CASE;
```

```
CASE
WHEN condition_1 THEN statements_1
WHEN condition_2 THEN statements_2
...
WHEN condition_n THEN statements_n
[ ELSE
  else_statements ]
END CASE;
```

 Si no se entra por ninguna opción y no hay parte ELSE se genera una excepción CASE_NOT_FOUND

Control iterativo

LOOP

[etiqueta] LOOP

conjunto de sentencias;

EXIT WHEN condición de salida

conjunto de sentencias;

END LOOP [etiqueta];

Control iterativo

WHILE

[etiqueta] WHILE condición de permanencia **LOOP**
conjunto de sentencias;
END LOOP [etiqueta] ;

Control iterativo

∞ Bucle FOR numérico

```
[etiqueta] FOR variable IN [REVERSE] rango_mínimo..rango_máximo  
LOOP  
    conjunto de sentencias;  
END LOOP [etiqueta]
```

Control iterativo

DECLARE

i **NUMBER**(3) :=1;

BEGIN

<pre>WHILE i<10 LOOP INSERT INTO tab VALUES(i); i:=i+1; END LOOP;</pre>	<pre>LOOP INSERT INTO tab VALUES(i); i:=i+1; EXIT WHEN i>9; END LOOP;</pre>	<pre>FOR i IN 1..9 LOOP INSERT INTO tab VALUES(i); END LOOP;</pre>
---	---	--

END;

Control iterativo

☞ Otras sentencias/instrucciones

- **EXIT [etiqueta]**. Abandona de forma incondicional la iteración actual de un bucle y transfiere el control al final del bucle o a un bucle etiquetado que lo envuelva.
- **EXIT WHEN**. Lo mismo que la anterior cuando se verifica la condición WHEN.
- **CONTINUE**. Abandona de forma incondicional la iteración actual de un bucle y transfiere el control a la siguiente iteración del bucle o a un bucle etiquetado que lo envuelva.
- **CONTINUE WHEN**. Lo mismo que la anterior cuando se verifica la condición WHEN.

Control iterativo

Ejemplo

DECLARE

```
s  PLS_INTEGER := 0;
i  PLS_INTEGER := 0;
j  PLS_INTEGER;
```

BEGIN

```
<<outer_loop>>
```

LOOP

```
  i := i + 1;
```

```
  j := 0;
```

```
<<inner_loop>>
```

LOOP

```
  j := j + 1;
```

```
  s := s + i * j; -- Sum several products
```

```
  EXIT inner_loop WHEN (j > 5);
```

```
  EXIT outer_loop WHEN ((i * j) > 15);
```

```
  END LOOP inner_loop;
```

```
END LOOP outer_loop;
```

```
DBMS_OUTPUT.PUT_LINE('The sum of products equals: ' || TO_CHAR(s));
```

END;

Control secuencial

- ∞ GOTO (no usar)
- ∞ NULL→indica de forma explícita inacción.

```
IF ratio>90 THEN  
    calcula(dni);  
ELSE  
    NULL;  
END IF;
```

SQL

SELECT

- La instrucción **SELECT ... INTO** se utiliza para traer valores de columnas de la base de datos
 - La sentencia **SELECT** **tiene que devolver una y sólo una fila**
 - Si devuelve más de una fila se genera la excepción **TOO_MANY_ROWS**.
 - Si no devuelve ninguna fila se genera la excepción **NO_DATA_FOUND**
 - Por cada elemento que se ponga en la parte *select* debe haber un elemento compatible en type en la parte *into*

```
SELECT select_item [, select_item ]...  
INTO variable_name [, variable_name ]...  
FROM table_name;
```

```
SELECT ename, deptno  
INTO v_nombre,v_depto  
FROM scott.emp  
WHERE empno=123;
```

SQL

∞ INSERT, DELETE y UPDATE

- Pueden ser incluidas dentro de la parte ejecutable de un bloque PL/SQL
- Pueden referenciar variables PL/SQL
- Si un UPDATE o un DELETE no afecta a ninguna fila no se produce error

DECLARE

X NUMBER;

Y CHAR;

BEGIN

X:=10;

Y:='prueba';

INSERT INTO temp VALUES (X,Y);

END;

Transacciones

∞ COMMIT [WORK]

∞ ROLLBACK [WORK]

∞ SAVEPOINT → punto de salvaguarda

- ROLLBACK [WORK] TO punto_de_salvaguarda;

Transacciones

∞ Ejemplo

```
BEGIN
  INSERT INTO tabla VALUES ('uno');
  SAVEPOINT A;
  INSERT INTO tabla VALUES ('dos');
  SAVEPOINT B;
  INSERT INTO tabla VALUES ('tres');
  SAVEPOINT C;
  .....
  IF x THEN
    ROLLBACK TO B;
  ELSIF y THEN
    ROLLBACK;
  ELSE
    ROLLBACK TO A;
  END IF;
  COMMIT;
END;
```

Cursores

- ✎ Almacenan los resultados de una consulta para ser procesados por otros comandos del bloque PL/SQL.
- ✎ Se emplean:
 - En sentencias SELECT que devuelvan más de una fila
 - Cuando se necesitan tratamientos fila a fila
- ✎ Los cursores pueden ser
 - Explícitos
 - Implícitos

Cursores explícitos

☞ Pasos para procesar un cursor explícito:

1. Declaración
2. Apertura para consulta
3. Recogida de los resultados en variables
4. Cierre

Cursores explícitos

Declaración

- Un cursor debe ser declarado antes de ser usado
- Siempre se declara en la zona DECLARE
- La consulta no debe tener la cláusula INTO
- Puede hacer referencia a variables ya declaradas
- Pueden contener parámetros que se declaran al definir el cursor

CURSOR nombre [lista_parámetros] **IS**
sentencia **SELECT** [cláusula **FOR UPDATE OF**] ;

DECLARE

```
CURSOR c_emp IS  
SELECT *  
FROM emp;
```

DECLARE

```
CURSOR c_emp (p_cod_d NUMBER) IS  
SELECT *  
FROM emp  
WHERE dept=p_cod_d;
```

DECLARE

```
a NUMBER;  
CURSOR c_emp IS  
SELECT *  
FROM emp  
WHERE cod>a;
```

Cursores explícitos

- Los cursores se recorren mediante bucles de cursor. Los bucles de tipo LOOP y WHILE requieren que el programador utilice las instrucciones OPEN, FETCH y CLOSE para procesar el cursor.

```
DECLARE
    v_empno scott.emp.empno%TYPE;
    v_ename scott.emp.ename%TYPE;
    CURSOR c1 IS SELECT empno,ename FROM scott.emp;
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO v_empno,v_ename;
        EXIT WHEN c1%NOTFOUND;
        .....
    END LOOP;
    CLOSE c1;
END;
```

Cursores explícitos

∞ Existe un bucle más sencillo: el FOR.

```
FOR variable IN cursor LOOP  
    ..  
END LOOP
```

∞ Dos particularidades del tipo FOR:

- La *variable* no es necesario declararla (se hace implícitamente y es de tipo `cursor%ROWTYPE`)
- No es necesario realizar explícitamente la apertura, la extracción de datos ni el cierre del cursor.

```
DECLARE  
    CURSOR c1 IS SELECT empno,ename FROM scott.emp;  
BEGIN  
    FOR i IN c1 LOOP  
        DBMS_OUTPUT.PUT_LINE(i.empno) ;  
    END LOOP ;  
END ;
```

Atributos de cursor explícito

- Existen una serie de atributos que pueden aplicarse sobre los cursores
 - **%FOUND.** TRUE si la última instrucción FETCH devolvió una fila.
 - **%NOTFOUND.** TRUE si la última instrucción FETCH NO devolvió una fila.
 - **%ISOPEN.** TRUE si el cursor está abierto
 - **%ROWCOUNT.** Número de filas extraídas del cursor hasta el momento

Cursores FOR UPDATE

- ✎ Cuando se quieren modificar las filas extraídas por un cursor se utiliza **SELECT FOR UPDATE**
 - `SELECT ...FROM...FOR UPDATE [OF columna, columna, ...] [NOWAIT]`
 - En la lista de columnas se especifica cuáles se van a modificar para limitar las filas que se van a bloquear
 - Con `NOWAIT` se devuelve un error si otra sesión tiene bloqueadas las filas
- ✎ Para hacer referencia al registro actual se puede usar **WHERE CURRENT OF *cursor***

```
DECLARE
    CURSOR c1 IS SELECT * FROM emp
                FOR UPDATE;

BEGIN
    FOR i IN c1 LOOP
        ...
        UPDATE emp set sal=incremento(sal)
            WHERE CURRENT of c1;
    END LOOP;
    COMMIT;
END;
```


Cursores implícitos

- Es posible utilizar un cursor sin definirlo previamente

BEGIN

FOR i **IN** (select * from scott.emp) **LOOP**

DBMS_OUTPUT.PUT_LINE(i.empno);

END LOOP;

END cursor_for_select;

END;

Cursores implícitos

- ✎ Siempre que se ejecuta una orden INSERT, DELETE, UPDATE o SELECT...INTO se crea un cursor implícito. El cursor recibe el nombre de SQL
- ✎ Sobre estos cursores se pueden utilizar los atributos de cursor

.....

```
UPDATE emp SET sal=sal*2
WHERE deptno='10';
IF SQL%ROWCOUNT = 0 THEN
```

... .



Cursores – Variables de cursor

- ✧ Una variable de cursor es una referencia a un cursor (puntero).
- ✧ El tipo de retorno debe ser de tipo registro
- ✧ Hay dos tipos:
 - Restringidas (Strong) (se conoce el tipo de retorno)
 - No restringidas (Weak) (no se conoce el tipo de retorno)
- ✧ Declaración
`TYPE nombre IS REF CURSOR [RETURN tipo_retorno];`
- ✧ Para asociar una variable con un cursor se utiliza OPEN-FOR



Cursores – Variables de cursor

DECLARE

```
TYPE t_cursor_emp IS REF CURSOR RETURN scott.emp%ROWTYPE;  
V_emp t_cursor_emp;  
TYPE registro IS RECORD (cod NUMBER, descrip VARCHAR2(100));  
V_Registro registro;  
TYPE t_cursor_reg IS REF CURSOR RETURN registro;  
TYPE t_cursor_reg2 IS REF CURSOR RETURN v_registro%TYPE;  
TYPE cursor_no_restringido IS REF CURSOR;
```

BEGIN

IF X THEN

```
OPEN v_emp FOR SELECT * FROM scott.emp WHERE sal>1200;
```

ELSIF Y THEN

```
OPEN v_emp FOR SELECT * FROM scott.emp WHERE job='CLERK';
```

ELSE

```
OPEN v_emp FOR SELECT * FROM scott.emp WHERE deptno='20';
```

END IF;

Cursores – Variables de cursor

DECLARE

```
TYPE tipo_cursor IS REF CURSOR ;  
v_cursor tipo_cursor;  
v_emp_rec scott.emp%ROWTYPE;  
v_dept_rec scott.dept%ROWTYPE;
```

BEGIN

```
OPEN v_cursor FOR SELECT * FROM scott.emp;  
LOOP  
    FETCH v_cursor INTO v_emp_rec;  
    EXIT WHEN v_cursor%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE('nombre = ' || v_emp_rec.ename );  
END LOOP;
```

CLOSE v_cursor;

--ahora hacemos lo mismo con otra select

```
OPEN v_cursor FOR SELECT * FROM scott.dept;  
LOOP  
    FETCH v_cursor INTO v_dept_rec;  
    EXIT WHEN v_cursor%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE('nombre depto=' || v_dept_rec.dname);  
END LOOP;
```

CLOSE v_cursor;

Advertencia

- ✧ Si vas a trabajar con muchas filas (>100) tanto en el SELECT ..INTO como en el OPEN FETCH debes usar el modo BULK. FOR ya hace uso de forma automática.
- ✧ Si vas a hacer una operación DML con muchas filas también debes hacer uso de BULK, mediante FORALL
- ✧ <https://docs.oracle.com/database/121/LNPLS/tuning.htm#LNPLS879>



Excepciones

- ✎ Los errores ocurridos durante la ejecución de la sección ejecutable de un bloque PL/SQL son llamados excepciones y se tratan en el gestor de excepciones
- ✎ Tipos de excepciones
 - Internas (tienen un código asignado. ORA-*n* errors)
 - Predefinidas (Internas que tienen un nombre asignado, además del código)
 - Definidas por el usuario (deben ser declaradas en la sección declarativa)



Excepciones - declaración

- La declaración de excepciones tiene lugar en la sección declarativa del bloque

Nombre **EXCEPTION**;

[**PRAGMA** **EXCEPTION_INIT**(nombre,-número);]

- Se puede asignar un número a una excepción definida por el usuario
- También se puede asignar un nombre a un código de error de Oracle

DECLARE

```
mi_error EXCEPTION; --declaración de excepción
```

```
columna_obligatoria_perdida_o_null EXCEPTION;
```

```
--asignación de un código de error a la excepción
```

```
PRAGMA EXCEPTION_INIT(mi_error,-21000);
```

```
--asignación de un nombre a un código de error de Oracle
```

```
PRAGMA EXCEPTION_INIT(columna_obligatoria_perdida_o_null,-1400);
```



Excepciones - predefinidas

☞ Algunas excepciones predefinidas:

DUP_VAL_ON_INDEX	INVALID_CURSOR
INVALID_NUMBER	LOGON_DENIED
NO_DATA_FOUND	NOT_LOGGED_ON
PROGRAM_ERROR	STORAGE_ERROR
TIMEOUT_ON_RESOURCE	TOO_MANY_ROWS
VALUE_ERROR	ZERO_DIVIDE
TRANSACTION_BACKED_OUT	ROWTYPE_MISMATCH
CURSOR_ALREADY_OPEN	ACCES INTO NULL
COLLECTION_IS_NULL	SUBSCRIPT_OUTSIDE_LIMIT
SUBSCRIPT_BEYOND_COUNT	



Excepciones – generación

- ✧ Cuando se produce un error se genera la excepción asociada.
- ✧ Las excepciones predefinidas e internas son generadas implícitamente por el sistema. Las definidas por el usuario deben ser generadas explícitamente mediante la instrucción RAISE
- ✧ Las predefinidas e internas también pueden generarse explícitamente mediante RAISE

```
RAISE mi_excepcion;  
RAISE NO_DATA_FOUND;  
RAISE; --propaga la excepción actual al nivel superior
```



Excepciones - generación

- ∞ También se puede emplear la función `RAISE_APPLICATION_ERROR` para crear excepciones con sus propios mensajes de error.

```
RAISE_APPLICATION_ERROR(número,mensaje,preservar_errores)
```

- El número debe estar entre `-20000` y `-20999`
- El mensaje puede ser de hasta 2048 caracteres
- Preservar errores es un valor booleano que indica si este error debe añadirse a los ya generados o debe sustituirlos



Excepciones - tratamiento

- El control de excepciones tiene lugar en los gestores

.....

EXCEPTION

WHEN nombre_de_excepción **THEN**
conjunto de sentencias;

WHEN nombre_de_excepción **OR** nombre_de_excepción **THEN**
conjunto de sentencias;

.....

WHEN OTHERS THEN
conjunto de sentencias;

END;



Excepciones – tratamiento

- ✎ El gestor OTHERS se encarga de cualquier excepción no tratada explícitamente
 - Debe ser el último gestor del bloque
 - Es conveniente definirlo en el bloque superior del programa
- ✎ Dentro de un gestor de excepciones podemos propagar esa excepción al nivel superior

```
WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN  
    RAISE;--propaga la excepción actual al nivel superior
```



Excepciones

☞ Ejemplo:

```
DECLARE
    a NUMBER(2) ;
BEGIN
    SELECT dept_no INTO a
    FROM dept
    WHERE dept_no=:B1.dept_no;
EXCEPTION
    WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN
        INSERT INTO Temp VALUES ('Departamento Incorrecto');
END;
```



Excepciones – propagación

∞ Propagación de excepciones

- Si la excepción se produce en la sección ejecutable:
 - Se trata en el gestor de excepciones (si tiene) y termina el bloque con éxito, pasando el control al nivel superior
 - Si no hay gestor de excepciones se propaga la excepción al nivel superior
- Si la excepción se produce en la sección declarativa:
 - Se propaga de forma inmediata al nivel superior
- Si la excepción se produce en la sección de excepciones:
 - Se propaga de forma inmediata al nivel superior



Excepciones – funciones

✎ Funciones usadas con excepciones:

- `SQLCODE` → devuelve el código de error que ha ocurrido (number)
- `SQLERRM` → devuelve la cadena de texto correspondiente al error ocurrido (varchar2). Máx. 512 bytes
- `DBMS_UTILITY.FORMAT_CALL_STACK` → devuelve la pila actual de llamadas (varchar2)
- `DBMS_UTILITY.FORMAT_ERROR_STACK` → devuelve la secuencia actual de errores (varchar2). Máx. 2000 bytes



Excepciones

∞ Ejemplo

```
DECLARE
    fatal EXCEPTION;
    ....
BEGIN
    ....
    IF campo IS NULL THEN RAISE fatal;
    ....
    EXCEPTION
        WHEN fatal THEN
            ....
        WHEN OTHERS THEN
            cod:=SQLCODE;
            mensaje:=SQLERRM;
            INSERT INTO errores VALUES (cod,mensaje);
            COMMIT;

END;
```



Paquetes

∞ Un objeto de la base de datos que agrupa lógicamente elementos que están relacionados:

- Procedimientos
- Funciones
- Cursores
- Tipos
- Variables (permiten variables privadas y “globales”)
- Excepciones



Paquetes

Constan de

○ **Especificación**

- Sección de declaración específica del paquete
- Declara elementos públicos.
- Elementos públicos son: procedimientos, funciones, variables, constantes, cursores, tipos, excepciones, ...

○ **Cuerpo**

- Cada declaración de cursor o subprograma realizado en la especificación del paquete tiene su correspondiente definición del cuerpo del paquete
- Puede además declarar y definir elementos privados
- Puede ser cambiado sin que afecte a la especificación del paquete (no es necesario compilarlo de nuevo)
- Cada sesión que referencia un elemento del paquete tiene su propia instancia del mismo (y su propia versión de estado)



Paquetes

∞ Constan de

○ **Objetos Públicos**

- Son declarados en la zona de especificación. Su ámbito es el esquema. Son visibles desde cualquier lugar del esquema

○ **Objetos Privados**

- Se declaran en la zona del cuerpo y sólo pueden ser empleados dentro del cuerpo del paquete

∞ **Privilegio de EXECUTE** sobre el paquete

- Permite al usuario acceder a cualquiera de los objetos públicos que hayan sido declarados en la especificación del paquete.



Paquetes

∞ Especificación

- Es necesario tener el privilegio del sistema de **CREATE PROCEDURE**
- Si se crea en un esquema diferente al propio es necesario el privilegio del sistema **CREATE ANY PROCEDURE**

```
CREATE [OR REPLACE] PACKAGE nombre {IS|AS}  
  especificación_procedimientos |  
  especificación_funciones |  
  declaración_variables |  
  definición_tipo |  
  declaración_excepciones |  
  declaración_cursores  
END [nombre];
```



Paquetes

∞ Cuerpo

- El nombre del cuerpo del paquete debe ser el mismo que el de la especificación del paquete

```
CREATE [OR REPLACE] PACKAGE BODY nombre {IS|AS}
    código_procedimientos (código) |
    código_funciones (código) |
    código_cursores (SELECT)
[BEGIN
    código_inicialización_paquete]
END [nombre];
```



Paquetes

∞ Inicialización de paquetes:

- Código que, en cada sesión, se ejecutará la primera vez que un usuario ejecute una función o procedimiento perteneciente al paquete
- Este código se incluye en el propio bloque PL/SQL al final del cuerpo del paquete.
- No dispone de su propia cláusula **end**; utiliza la cláusula **end** del cuerpo del paquete.



Paquetes

☞ Beneficios:

- **Minimiza conflictos de nombres dentro de un esquema**
 - Para referirnos a ellos:
 - **Esquema.paquete.objeto**
- **Simplifica la seguridad**
 - GRANT EXECUTE afecta a todo el paquete, no sólo a una función o procedimiento
- **Permiten la sobrecarga de procedimientos y funciones**



Disparadores

- ✧ **Subprogramas que se ejecutan automáticamente cuando ocurre un suceso de disparo (orden DML, orden DDL, evento de la instancia)**
- ✧ La acción de un disparador puede causar que se dispare otro



Disparadores

- ✎ Existen tres tipos de disparadores
 - **Disparadores DML** (responden a INSERT, UPDATE o DELETE sobre tablas o vistas)
 - **Disparadores de sustitución** (se definen sobre vistas) (Instead of)
 - **Disparadores del sistema** (responden a eventos del sistema-database triggers- y a instrucciones DDL-schema triggers-)
- ✎ Permisos necesarios para crear un disparador:
 - Propietario de la tabla/vista – Privilegio ALTER o ALTER ANY TABLE/VIEW
 - Privilegio CREATE TRIGGER o CREATE ANY TRIGGER



Disparadores

Sintaxis

```
CREATE [OR REPLACE] TRIGGER esquema.nombre  
  {BEFORE|AFTER|INSTEAD OF} evento_de_disparo  
  cláusula_de_referencia  
  [WHEN condición]  
  [FOR EACH ROW]  
  Bloque PL/SQL
```



Disparadores

- ✎ Activación y Desactivación de triggers:
 - ALTER TRIGGER nombre DISABLE;
 - ALTER TRIGGER nombre ENABLE;
 - ALTER TABLE nombretabla DISABLE ALL TRIGGERS;
- ✎ Cuando un trigger está **desactivado** no se ejecuta aunque ocurra el evento de disparo
- ✎ Borrar disparadores:
 - DROP TRIGGER nombre;



Disparadores DML

- ∞ Un disparador DML puede ejecutarse (temporalidad)
 - BEFORE
 - AFTER
- ∞ De una instrucción (evento de disparo)
 - INSERT
 - UPDATE
 - DELETE



Disparadores DML

∞ Pueden ser:

- **A nivel de fila (for each row)**
 - Se ejecutan **una vez** (before o after) **por cada fila afectada** por la instrucción DML que desencadenó el trigger
 - Son el tipo más común
- **A nivel de instrucción (sentencia)**
 - Se ejecutan **una vez** (before o after) **por cada instrucción DML**
 - Es el tipo de disparador predeterminado que se crea con CREATE TRIGGER



Disparadores DML

∞ Sintaxis disparador DML

```
CREATE [OR REPLACE] TRIGGER esquema.nombre  
  {BEFORE|AFTER} {[OR]DELETE| [OR] INSERT| [OR]  
    UPDATE [OF columna]}  
  ON esquema.tabla  
  [FOR EACH ROW] [WHEN condición]  
Bloque PL/SQL
```



Disparadores DML

✎ **Identificadores de correlación** (sólo disponibles en disparadores a nivel de fila). Un trigger a nivel de fila puede acceder a los datos de la fila (registro) que está procesando a través de los identificadores de correlación (también llamados pseudoregistros):

- **:NEW** // Hace referencia al nuevo valor de una fila(en un DELETE no está definido, es NULL)
- **:OLD** // Hace referencia al valor antiguo de una fila(en un INSERT no está definido, es NULL)

...

```
IF :new.sal>:old.sal THEN  
    --el salario se ha incrementado...  
ELSIF :new.sal<:old.sal THEN  
    --el salario se ha decrementado...  
END IF;
```



Disparadores DML

- ✧ La cláusula WHEN permite especificar una condición SQL que es evaluada para cada fila afectada por la instrucción que desencadena el trigger. Si el resultado de la evaluación es TRUE entonces el trigger se ejecuta
- ✧ Válido únicamente para los triggers a nivel de fila (for each row)

```
CREATE OR REPLACE TRIGGER tr_1
  BEFORE INSERT OR UPDATE OF salario ON empleados
  FOR EACH ROW
  WHEN (new.salario>1000)
BEGIN
...
END;
```



Disparadores DML

∞ **Predicados condicionales.** Sirven para averiguar qué sentencia SQL disparó el trigger

- `UPDATING`→`TRUE` si el evento que generó el trigger es `UPDATE`, `FALSE` en otro caso
- `UPDATING ('columna')`→`TRUE` si el evento que generó el trigger es un `UPDATE` sobre la columna indicada, `FALSE` en otro caso
- `INSERTING`→`TRUE` si el evento que generó el trigger es `INSERT`, `FALSE` en otro caso
- `DELETING`→`TRUE` si el evento que generó el trigger es `DELETE`, `FALSE` en otro caso



Disparadores DML

Orden de ejecución:

1. INSERT/DELETE/UPDATE es pasado a Oracle
2. Ejecución de TODOS los disparadores BEFORE a nivel de sentencia
3. Por cada fila afectada por la sentencia SQL:
 - 3.1 Ejecución de TODOS los disparadores BEFORE a nivel de fila
 - 3.2 Ejecución de la instrucción y comprobación de las restricciones de integridad referencial
 - 3.3 Ejecución de TODOS los disparadores AFTER a nivel de fila
4. Completar la comprobación de restricciones de integridad referencial para sentencias internas
5. Ejecución de todos los disparadores AFTER a nivel de sentencia
6. Devolver control a la aplicación

- a. BEFORE
- b. BEFORE ROW
- c. AFTER ROW
-
- d. BEFORE ROW
- e. AFTER ROW
- f. AFTER



Disparadores DML

- ✎ Si hay dos disparadores con el mismo evento → orden de ejecución aleatorio
- ✎ No se puede acceder a una **tabla mutante** (aquella que está siendo modificada por una instrucción DML o a través de **ON DELETE CASCADE**). El compartimiento varía de unas versiones a otras.



Disparadores DML

∞ Aplicaciones de los Disparadores

- Mantener campos calculados o derivados
- Forzar al cumplimiento de reglas de negocio
- Implementar reglas de integridad referencial complejas que no pueden definirse con restricciones básicas (check, unique,...)
- Mejorar el rendimiento con valores basados en auditorías
- Mantener tablas replicadas



Disparadores DML

∞ Diferencias entre triggers y procedimientos almacenados:

- Los disparadores están asociados a tablas
- Se invocan de manera implícita, nunca explícitamente
- Las sentencias COMMIT, ROLLBACK o SAVEPOINT no se permiten en los disparadores
- Los usuarios no necesitan privilegio EXECUTE sobre los disparadores



Paquetes – DBMS_OUTPUT

- ✧ DBMS_OUTPUT permite enviar mensajes a un buffer (que normalmente se muestra por pantalla). Es especialmente útil para depuración.
- ✧ Para que se muestre por pantalla en SQL+ es necesario invocar a `set serveroutput on`
- ✧ Es equivalente a `DBMS_OUTPUT.ENABLE();`
- ✧ Los mensajes se envían al buffer una vez terminada la ejecución del programa



Paquetes – DBMS_OUTPUT

🌀 Los métodos más importantes:

ENABLE ENABLE(buffer)	Activa el buffer. Se le puede indicar el tamaño del buffer (por defecto: 20000)
NEW_LINE	Escribe un fin de línea en el buffer
PUT(cadena)	Escribe una cadena(varchar2) en el buffer. NO añade fin de línea
PUT_LINE	Escribe una cadena(varchar2) en el buffer añadiendo un fin de línea
GET_LINE(lin,estado)	Lee un línea del buffer. En <i>lin</i> devuelve la línea leída. Si estado es 0 quedan más líneas, si es 1 no quedan más

Pseudocolumnas en Oracle

- ✎ Una pseudocolumna se comporta como si fuese una columna de una tabla, pero en realidad no está almacenada en la tabla.
- ✎ La única operación que se puede realizar sobre ellas es consultar su valor (no se pueden insertar, borrar ni actualizar)

- **ROWNUM**

- Devuelve el número que indica el orden en el que Oracle devuelve los datos en una consulta.
- El primer número es el 1
- Se puede utilizar, por ejemplo, para limitar el número de registros que recupera una SELECT
- Si en la misma consulta hay un ORDER BY tras el ROWNUM la ordenación se realiza tras asignar el número de ROWNUM. Se puede forzar a que la asignación del número de ROWNUM se aplique después de realizar la ordenación poniendo el ORDER BY en una subquery y el ROWNUM en la consulta de nivel superior.

Fechas en Oracle

☞ Tipo **DATE**

- Almacena información de **fechas y horas**: año, mes, día, hora, minuto y segundo.
- Si se especifica una fecha sin hora se toma como hora por defecto 00:00:00. Si se especifica una hora sin fecha se toma por defecto el primer día del mes actual del año actual
- Para especificar fechas se puede hacer
 - Mediante una cadena de caracteres
 - Por cualquier formato admisible empleando la función TO_DATE



Fechas en Oracle (II)

∞ Funciones relacionadas con fechas

○ **ADD_MONTHS (fecha, meses)**

- Suma los meses a la fecha dada
- Ejemplo: `SELECT ADD_MONTHS ("31/01/2005",1) FROM tabla` -> 28/02/2005

○ **LAST_DAY (fecha)**

- Devuelve el último día del mes que se le pasa como parámetro
- Ejemplo: `SELECT LAST_DAY ("05/01/2005") FROM tabla` -> 31/01/2005

○ **MONTHS_BETWEEN (fecha1, fecha2)**

- Devuelve el número de meses entre dos fechas
- Ejemplo: `SELECT MONTHS_BETWEEN ("01/03/05", "01/09/05") FROM tabla` -> -6

○ **NEXT_DAY (fecha, día de la semana)**

- Devuelve la fecha del siguiente día de la semana a partir de la fecha dada
- Ejemplo: `SELECT NEXT_DAY ("15/05/05", "MARTES") FROM tabla` -> 17/05/05



Fechas en Oracle (III)

🔗 Funciones relacionadas con fechas

○ **ROUND (fecha [,unidad])**

- Devuelve una fecha dada redondeada a la unidad que se le especifique. Si no se especifica nada se toman días
- Ejemplo1: `SELECT ROUND('23/10/1993','YYYY') FROM tabla->01/01/94`
- Ejemplo1: `SELECT ROUND('12/11/2002','MM') FROM tabla->01/11/02`

○ **SYSDATE**

- Devuelve la fecha y la hora actual del servidor. No recibe ningún argumento
- Ejemplo: `SELECT TO_CHAR(SYSDATE,'DD/MM/YYYY HH24:MI:SS') FROM tabla->01/02/2002 12:35:15`

○ **TRUNC (fecha [,unidad])**

- Devuelve una fecha dada truncada a la unidad que se le especifique. Si no se especifica unidad se toman días.
- Ejemplo: `SELECT TRUNC(TO_DATE('02/05/2005','DD/MM/YYYY'),'YYYY') FROM tabla->01/01/05`
- Ejemplo: `SELECT TRUNC(TO_DATE('02/05/2005','DD/MM/YYYY'),'DD') FROM tabla->02/05/05`



Fechas en Oracle (IV)

∞ Funciones de conversión relacionadas con fechas

○ **TO_CHAR** (fecha[, formato[, 'parametro_nls']])

- Convierte una fecha de tipo DATE en un tipo de dato VARCHAR2 con el formato especificado como argumento
- Ejemplo: `SELECT TO_CHAR (SYSDATE, 'Day, DD/Month/YYYY') from tabla -> Viernes, 01/Febrero /2002`
- Ejemplo: `SELECT TO_CHAR (SYSDATE, 'DAY, DD/Month/YYYY', 'NLS_DATE_LANGUAGE = American') from dual -> TUESDAY , 10/December /2013`

○ **TO_DATE** (cadena[, formato[, parametros_nls]])

- Convierte una cadena de caracteres de tipo CHAR o VARCHAR2 a un dato de tipo DATE
- Ejemplo: `SELECT TO_DATE( 'January 15, 1989, 11:00 A.M.', 'Month dd, YYYY, HH:MI A.M.', 'NLS_DATE_LANGUAGE = American') FROM DUAL;`



Valores nulos en Oracle

- ✎ Cuando una columna no tiene valor se dice que tiene valor null
- ✎ El **resultado de evaluar el null es indeterminado** lo que puede causar muchos problemas.
 - Ej. if (null=0) puede ser cierto o falso dependiendo del momento
- ✎ OPERADORES: IS NULL e IS NOT NULL
- ✎ FUNCION NVL (expresión, expresión_de_reemplazo)
 - Si la *expresión* es null devuelve la *expresión_de_reemplazo* segunda, y en caso contrario devuelve *expresión*.
 - Las expresiones pueden ser de cualquier tipo. Si son de distinto tipo Oracle transforma el tipo de la segunda al de la primera. El tipo de valor devuelto siempre es el de la primera.
 - example: `SELECT empno, NVL (TO_CHAR(COMM), 'SIN COMISIONES') FROM table`

